Institute of Geographical Information Systems

CS-250 - Data Structures and Algorithms LAB

Semester: Spring 2026	**Class: SCEE-IGIS - 2024**
Name: Ali Nawaz	**CMS ID : 00000526123**
Submitted to: Ma'am Alvina Anjum	**Due Date: April 22, 2026**

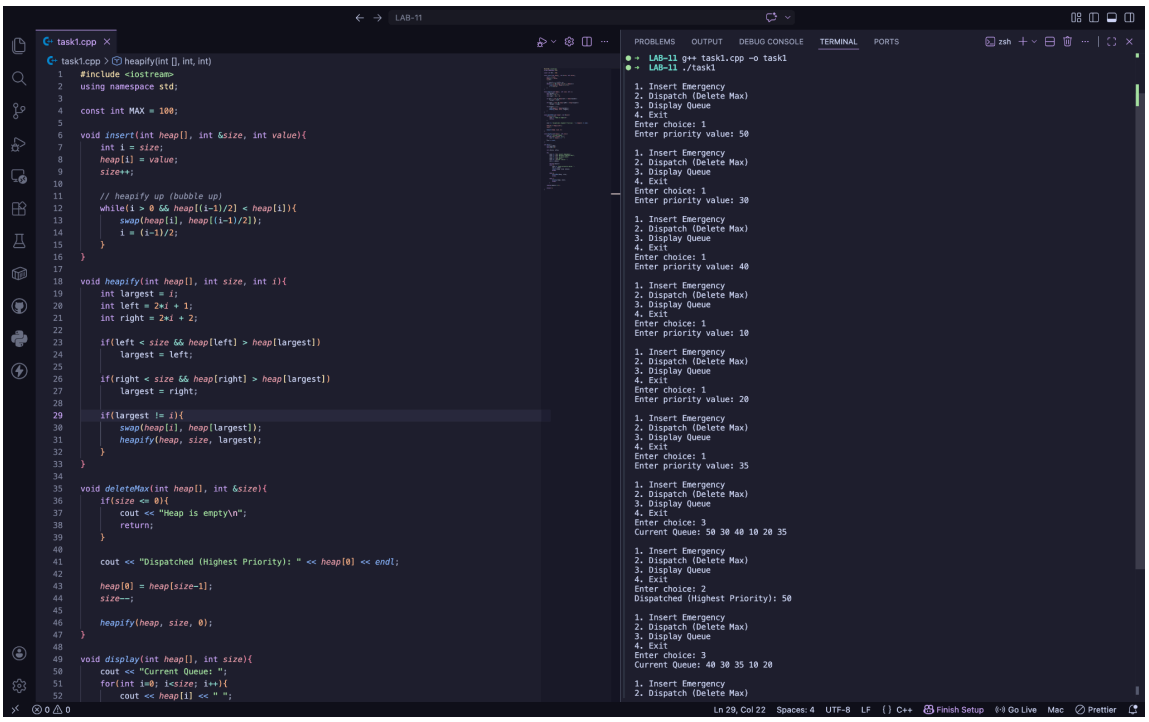
LAB 11: Heaps

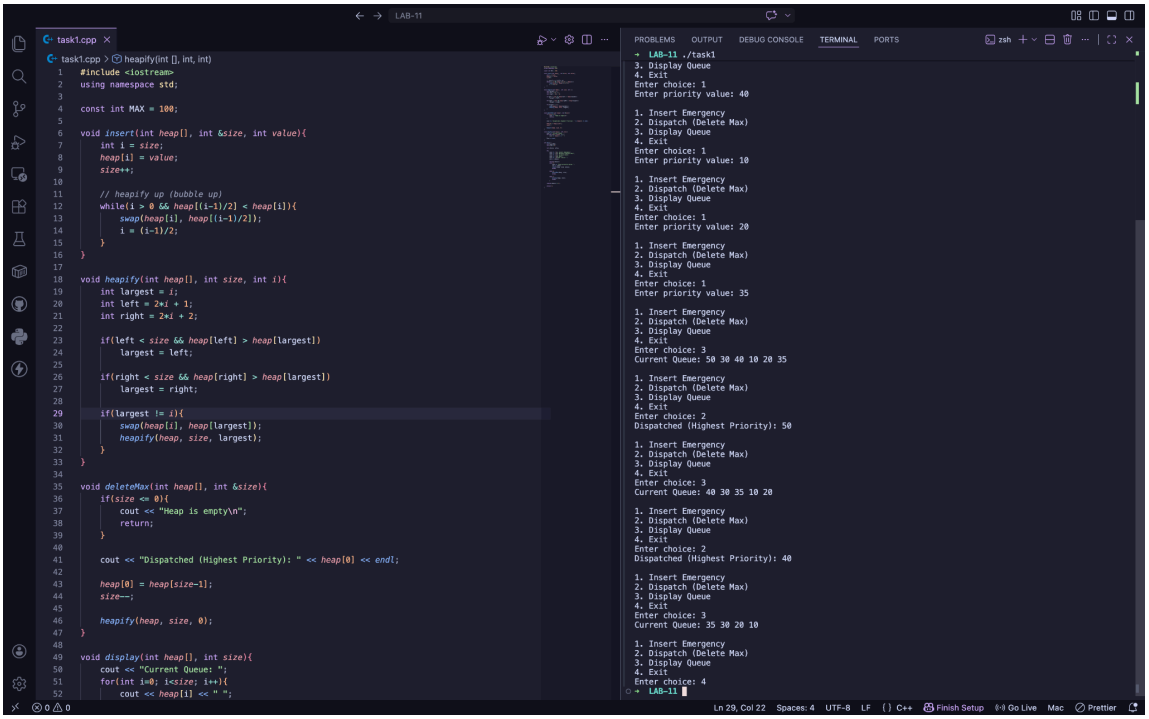
Task 1: (Core Task – CLO3)
Design a C++ program to simulate the emergency dispatch system.
Requirements:

- Build a Max-Heap using an array
- Implement the following operations:
  - Insert a new emergency request (priority value)
  - Dispatch (delete max priority)
  - Display current queue
  - Exit system
- Functions to Implement:
  - void insert(int heap[], int &size, int value);
  - void deleteMax(int heap[], int &size);
  - void heapify(int heap[], int size, int i);
  - void display(int heap[], int size);

Conceptual Link

- The Max-Heap ensures O(log n) insertion and deletion
- Root always contains the highest priority emergency





Task 2

Q: Explain why a Max-Heap-based priority queue is more suitable for this emergency system than:

- A simple array
- A linked list

Support your answer with time complexity analysis.

Ans: Max-Heap is more efficient because it allows insertion and deletion in O(log n) time, while arrays and linked lists require O(n) time to find the maximum.

Task 3: The emergency response system needs to generate a **daily report of handled cases sorted by priority** (highest priority first).

You are required to write a **C++ program** to sort the given list of emergency priority values using **any sorting algorithm of your choice** (e.g., Quick Sort, Merge Sort, etc.).

Input

An array of integer priority values:

int priorities[] = {45, 20, 60, 10, 75, 30};

Output

Sorted priorities in **descending order**:

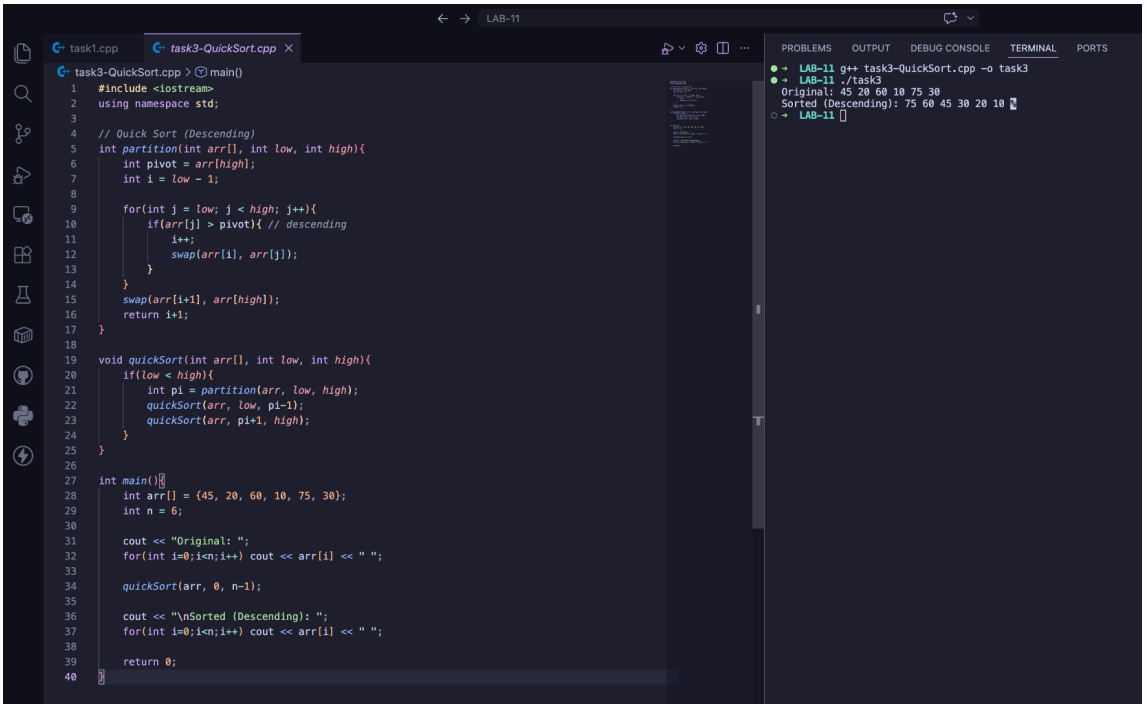
75 60 45 30 20 10

Requirements

- Implement the sorting algorithm **from scratch** (no built-in sort functions)
- Display:
  - Original array
  - Sorted array
- Clearly mention the algorithm used in comments

Function Prototype (Example)

void sortPriorities(int arr[], int n);




```cpp
#include <iostream>
using namespace std;

// Quick Sort (Descending)
int partition(int arr[], int low, int high){
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++){
        if(arr[j] > pivot){ // descending
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[high]);
    return i+1;
}

void quickSort(int arr[], int low, int high){
    if(low < high){
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi-1);
        quickSort(arr, pi+1, high);
    }
}

int main(){
    int arr[] = {45, 20, 60, 10, 75, 30};
    int n = 6;

    cout << "Original: ";
    for(int i=0;i<n;i++) cout << arr[i] << " ";

    quickSort(arr, 0, n-1);

    cout << "\nSorted (Descending): ";
    for(int i=0;i<n;i++) cout << arr[i] << " ";

    return 0;
}
```

Task 4: Optional – CLO4/CLO6

Q: Explain how **Dijkstra's Algorithm** uses a Min-Heap in graph-based emergency routing systems.

Ans: Dijkstra uses a Min-Heap to always pick the node with the smallest distance first. This helps find the shortest path efficiently in O(log n) time per operation.